

Операционная система Linux: Основы работы с командной строкой



Андрей
Тряпичников



Андрей Тряпичников

Системный администратор

Одноклассники



План занятия

1. [Виртуальная файловая система Linux и структура каталогов](#)
2. [Немного о пользователях и доступах](#)
3. [Интерфейс командной строки](#)
4. [Итоги](#)
5. [Домашнее задание](#)



Виртуальная файловая система Linux и структура каталогов



VFS: Виртуальная файловая система

Виртуальная файловая система — это уровень абстракции над реализациями файловых систем, удобное представление их для процессов.

По сути это **дерево каталогов**, единая иерархия, в которой процессы могут обращаться к данным, хранящимся на диске, в памяти, на удалённых серверах и т.д.

VFS: Виртуальная файловая система

В отличие от Windows, где каждое устройство представлено независимой иерархией (диск C:\, D:\ ...), VFS предполагает наличие **единой иерархии, начинающейся с корневого каталога (root, /)** и подключение (**монтирование**) файловых систем в каталогах внутри этой иерархии.

Вспомним один из принципов POSIX и LSB: **всё в системе представлено в виде файла**. Поэтому монтируются не только каталоги с данными, но и виртуальные представления устройств, процессов, свойств системы.

Иерархия каталогов Linux

```
/ ..... корневой каталог
|-- bin ..... исполняемые файлы
|-- lib ..... библиотеки
|-- etc ..... конфигурационные файлы
|-- root ..... домашний каталог пользователя root
|-- home ..... домашние каталоги пользователей
|   |-- linus
|   |-- patrick
|   `-- ian
|-- dev ..... представление в фс устройств
|-- proc ..... представление в фс процессов
|-- sys ..... представление в фс прочих системных
|               параметров и состояний
...
```

Иерархия каталогов Linux

```
...
|-- tmp.....временные файлы и директории
|-- usr.....дополнительная иерархия
|   |-- bin.....- исполняемые файлы
|   |-- lib.....- библиотеки
|   `-- share.....- ресурсы (локализация, документация и т.д.)
|-- var.....различные изменяющиеся файлы
|   |-- log.....- логи (журналы событий)
|   `-- cache.....- кэш (временное хранилище данных)
`-- opt.....самостоятельные иерархии каталогов
    `-- jdk-16.0.2.....- приложение с самостоятельной
                           иерархией каталогов
```




Относительные и абсолютные пути

У любого процесса (включая процесс командной строки) есть текущая (рабочая) директория. Путь до файла может быть абсолютным и относительным (относительно рабочего каталога). **Абсолютный путь всегда начинается с /.**



Особые директории и синонимы

Любая директория (даже корень /) всегда содержит две особые директории:

- . текущая директория
- .. директория уровнем выше

Кроме того, абсолютный путь домашних каталогов доступен по алиасам:

- ~ домашняя директория текущего пользователя
- ~user домашняя директория пользователя user

Особые директории и синонимы

Например, для рабочей директории `/var/tmp` и пользователя `ian`:

```
./opt    → /var/tmp/opt  
../opt   → /var/opt  
~/opt    → /home/ian/opt
```

Использование директорий `.` и `..` (но не `~`) допустимо и в абсолютных путях:

```
/var/tmp/./opt    → /var/tmp/opt  
/var/tmp/../opt   → /var/opt  
/var/tmp/~opt     → /var/tmp/~opt
```

Символьные и жёсткие ссылки

Иногда нужно сослаться на путь до каталога или файла. Продублировать адрес в файловой системе, не копируя данные. Для этого применяют **ссылки**.

Ссылки бывают **символьные** (мягкие) и **жёсткие**.

- символьная ссылка указывает на путь до цели
(магазин находится в том же здании, что и кафе)
- жёсткая ссылка является ещё одним адресом цели,
равноправным исходному
(до магазина можно дойти по улицам Ленина и Лесной)



Интерфейс командной строки

CLI: интерфейс командной строки

Существует два вида интерфейсов:

- Graphic User Interface (GUI), графический интерфейс
- Command Line Interface (CLI), **командная строка** (или **консоль**).

Преимущества командной строки:

- информативность,
- гибкость,
- примитивность,
- надёжность,
- экономия системных ресурсов.

Синтаксис команды

Синтаксис большинства команд **включает** в себя саму **команду** и **аргументы**, обычно аргументы делят на две категории: **опции** (их ещё называют **ключами**) и собственно **аргументы**. Опции модифицируют поведение команды, являются своего рода настройками и начинаются с **-** (короткой чертой). Аргументы обычно указывают на цели, с которыми команда оперирует. Например:

```
ls -lh -d --all . /tmp /opt
```

ls	команда
-lh -d --all	опции (обратите внимание, что они могут «склеиваться» или быть «длинными»)
. /tmp /opt	аргументы (цели)

Встроенная документация

- `man команда` — открыть документацию по команде
 - как читать:
 - [аргументы] в квадратных скобках **опциональны**,
 - аргументы без квадратных скобок **обязательны**,
 - `-o`, `--option` — взаимозаменяемые короткие / длинные опции;
- `команда --help`;
- `команда -h`
 - есть не для всех команд (а у некоторых вызывается иначе)
 - нет единого стандарта оформления.

Работа с текстом

- **echo** *текст* — вывести текст в консоль;
- **cat** [*цель*] — вывести содержимое файла (цели) в консоль;
- **head** [*цель*] — показать **первые** 10 строк файла
 - -15 — в качестве опции можно передать количество строк;
- **tail** [*цель*] — показать **последние** 10 строк файла
 - -15 — в качестве опции можно передать количество строк;
- **grep** *условие* [*цель*] — вывести строки, содержащие *условие*;
- **less** [*цель*] — интерактивный просмотр текста.

Стандартные потоки ввода-вывода

Поток номер 0 (STDIN) зарезервирован для *чтения команд* пользователя или *входных данных*.

```
command < filename
```

Поток номер 1 (STDOUT) зарезервирован для *вывода данных*, как правило текстовых.

```
command > filename или command >> filename
```

Поток номер 2 (STDERR) зарезервирован для вывода *диагностических и отладочных сообщений* в текстовом виде.

```
command 2> filename или command 2>> filename
```

В случае с потоками 1 и 2 один символ **>** означает **перезапись**, удвоенный **>>** — **дозапись**.

Конвейер (пайп)

Поскольку у любого процесса есть стандартные потоки ввода и вывода, их можно **перенаправлять** один в другой (вывод одной команды на ввод другой). Делается это с помощью, так называемого, **пайпа** или **конвейера**, в синтаксисе командной строки обычно представленного в виде **одной вертикальной черты** |:

```
command1 | command2
```

На слайдах с примерами команд в ряде инструментов цель помечена как необязательная, это потому, что многие команды для работы с текстом способны принимать его на стандартный ввод, например:

```
ls -lh /var/log | grep root
```

Работа с файлами и каталогами

- **pwd** — вывести рабочую директорию;
- **cd [цель]** — перейти в каталог
 - если не указывать цель — в домашний каталог пользователя;
- **mkdir цель** — создать каталог;
- **touch цель** — создать пустой файл;
- **ls [цель]** — просмотр содержимого каталога
 - если не указывать цель — просмотр содержимого рабочего каталога
 - **-l** — длинный (подробный) вывод
 - **-a** — отображение скрытых (начинающихся с точки) файлов и каталогов
 - **-h** — «человекочитаемый» вывод размеров файлов;

Работа с файлами и каталогами

- **ср** *ист[ист] цель* — копирует *источник* в *цель* (если источников несколько, то *цель* должна быть каталогом)
 - -R — рекурсивно (если в качестве источника каталог — копировать его и его содержимое)
 - -V — вывести подробности о выполняемых операциях;
- **mv** *ист[ист] цель* — перемещает исходный файл в целевой
 - -V — вывести подробности о выполняемых операциях;
- **ln -s** *ист цель* — создать символическую ссылку *цель* на источник *ист*
 - для создания жёсткой ссылки нужно выполнить команду без -s, но жёсткие ссылки на практике применяются очень редко;

Работа с файлами и каталогами

- **rm** *цель* — удалить файл
 - -R — рекурсивно (если нужно удалить каталог вместе с содержимым)
 - -v — вывести подробности о выполняемых операциях;
- **rmdir** *цель* — удалить пустую директорию
 - -v — вывести подробности о выполняемых операциях;

Работа с файлами и каталогами

- **tar** — работать с архивом формата tar (tape archiver)
 - команда по историческим причинам чувствительна к порядку аргументов, по умолчанию работает с STDIN / STDOUT
 - **-f *цель*** — работать с файлом, указанным в качестве цели
 - **-c** — создать (Create) архив
 - **-x** — распаковать (eXtract) архив;
- **find *[цель]*** — рекурсивно найти и вывести в консоль список файлов и директорий внутри цели
 - подобно tar обладает своеобразной чувствительностью к порядку аргументов
 - использует длинные опции, но с одним «-».

Пользователь root и повышение привилегий

Все современные операционные системы работают в многопользовательском режиме, и Linux не исключение. В Linux существует специальный пользователь-администратор (суперпользователь) — **root**.

Ряд вещей (но **далеко не все!**) необходимо осуществлять от имени суперпользователя. Например, управление пакетами. Для этого применяются два инструмента:

sudo более современный и гибкий, но по умолчанию установлен не везде.

su - более универсально доступный, но менее удобный.

Работа с пакетным менеджером

- **dnf** — пакетный менеджер Red Hat, CentOS, Fedora и многих других rpm-дистрибутивов, стандартные команды:
 - **dnf update** — обновить установленные пакеты
 - **dnf search *цель*** — искать пакет
 - **dnf install *цель* [*цель*]** — установить пакеты
 - **dnf remove *цель* [*цель*]** — удалить пакеты
- **rpm** — пакетный менеджер, отвечающий за непосредственную работу с пакетами:
 - **rpm -i** — установить пакет из файла
 - **rpm -q** — опросить локальную базу пакетов (напр, **-a**, **-l**)

Текстовые редакторы: **vi** и **nano** и **mcedit**

GNU nano — **простой в управлении** текстовый редактор, пригодный для небольших правок.

mcedit — текстовый редактор, пригодный для небольших правок.

vim — **модальный** текстовый редактор, предназначенный для углубленной работы. Имеет несколько режима работы: модальный, для перемещения по тексту, поиска и т.д., режим ввода, для ввода текста, визуальный для выделения текста и работы с ним, и другие.

vi входит в пакет **coreutils**, и следовательно, присутствует в любой системе на базе Unix. Является предшественником **vim** и обладает очень схожим функционалом.



Итоги

Итоги

Сегодня мы узнали:

- что такое виртуальная файловая система и как выглядит структура директорий в linux;
- в чём различия между абсолютным и относительным путём;
- как обращаться к текущей директории и переходить на уровень выше;
- чем хороша командная строка и какой функционал она предоставляет системному администратору;
- какие инструменты у нас есть для манипулирования файлами и текстом в командной строке;
- кто такой root, и что он может.





Домашнее задание

Домашнее задание

Давайте посмотрим ваше [домашнее задание](#).

- Вопросы по домашней работе задавайте **в чате**.
- Задачи можно сдавать **по частям**.
- Зачёт по домашней работе проставляется после того, как **приняты все задачи**.

**Задавайте вопросы и
пишите отзыв о лекции!**

Андрей Тряпичников

a.tryapichnikov@gmail.com